

docloom: LLM to Documents, Instantly

Validated IR, Deterministic Renderers, 6 Output Formats

docloom project — 2026-07-21

docloom: LLM to Documents, Instantly

Validated IR + Deterministic Renderers = 6 Native Formats

From Raw LLM Output to Production Files

- LLM outputs are unstructured and unpredictable
- Manual formatting is error-prone and time-consuming
- Renderers need consistent, validated input
- Quality checks should happen before any file is written
- Users need native, editable formats—not rasterized images

Validated IR, Deterministic Renderers

- LLM emits schema-validated JSON (Pydantic IR)
- Deterministic linter checks layout, citations, contrast
- Findings loop back to model for self-correction
- Deterministic renderers: same IR = same output, always

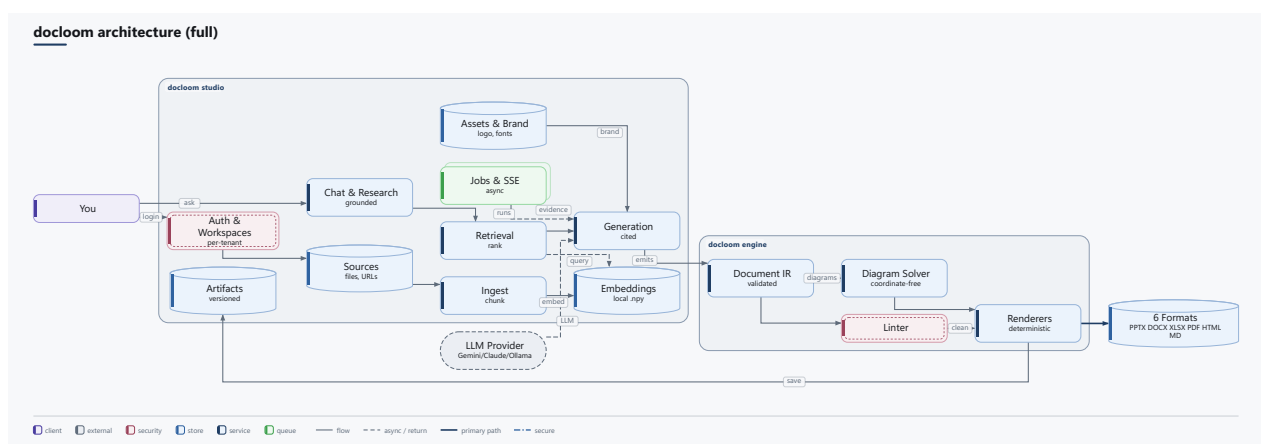
The IR is deliberately non-recursive with plain tagged unions—shaped to survive real LLM structured-output constraints.

Deliver the same file on every render

100%

Byte-identical renders

docloom architecture, end to end



The full studio + engine architecture: auth, retrieval, jobs, generation, versioned artifacts, and deterministic rendering.

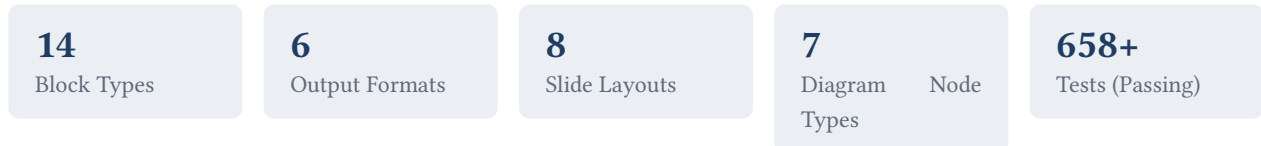
- Sources are chunked, embedded, and ranked, so only the most relevant evidence reaches the model

- The model emits validated IR, never files; the linter checks layout and citations before any render
- Deterministic renderers turn one IR into all six formats with no lossy conversion

Follow sources from ingest to shipped files

1. Ingest sources
2. Retrieve evidence
3. Generate IR
4. Lint findings
5. Render files

Count What Matters: 14 Block Types, 6 Formats, 658+ Tests



Generate Six Native Formats from One IR

- PPTX: native editable charts, autofit text, native shapes
- DOCX: hardened tables, repeating headers, fixed column widths
- XLSX: typed cells, formulas, number formats
- PDF: Typst compiled in-process, no LaTeX needed
- HTML: semantic markup with WCAG 2.x contrast
- Markdown: clean, readable prose

Build documents from families of typed blocks

- Text & headings
- Tables & charts
- Stats & callouts
- Diagrams & images
- Code & quotes

Turn Source Data into Grounded Artifacts

Add Sources

- Upload file, URL, or paste text
- Free web research agent included
- Hybrid BM25 + dense retrieval
- Automatic chunking at 1000 chars

Generate Artifacts

- Chat with grounded, cited answers
- 5 one-click guides (briefing, FAQ, timeline)
- 6 artifact kinds: deck, doc, sheet, diagram, infographic, podcast

Keep Text Legible at Any Scale

- Measured autofit: real text extents via FreeType against actual fonts
- WCAG 2.x contrast ratio checking: 4.5:1 for normal text
- Diagram legibility: labels never truncated, step down detail instead
- Table hardening: fixed widths, repeating headers, split prevention
- Geometric QA: off-slide detection, overlap checks, font sprawl

Design principle: lint before render. Only error severity blocks a render; warnings help improve.

Local-First, Offline-Ready, Multi-Tenant-Safe

- Runs fully offline against a local model (ollama, llama.cpp, etc.)
- SQLite default, Postgres for scale
- Session tokens stored only as SHA-256 hashes
- Cross-tenant access returns 404, not 403
- Secrets encrypted at rest with Fernet

One Command to Launch

Windows (PowerShell)

```
studio.ps1
```

macOS / Linux / Git Bash

```
studio.sh
```

- Installs dependencies, builds frontend, opens browser
- Default: `http://127.0.0.1:8899`
- Prerequisites: uv, Node 22+, npm

Set Up Providers, Storage, and Secrets

- Generation: ollama, llama-server, lmstudio, openai, anthropic, gemini
- Storage: SQLite (default) or Postgres via `DOCLOOM_DB_URL`
- Secrets: Fernet encryption, masked in API responses
- Optional extras: [pdf,diagrams] for engine; [podcast,ingest] for studio
- Settings resolve: per-user → global → built-in defaults

Design Principles: Built for Reliability

- Validated IR, not code: LLM output is data, never executable
- Deterministic renderers: same input = same output, always
- Lint before render: findings can loop back to model
- Nothing authored is lost: labels step down, never truncate
- Graceful degradation: optional deps improve output, never crash

Open Source, Local-First, Production-Ready

MIT License • Python 0.2.0 • 658+ Tests • Full-Stack SPA